

Mashudu Mavhila

Day 1 Mini Report

Kaggle Submission Format and Validation Setup

For Day 1, my responsibility as Member 4 was to prepare the Kaggle submission structure and set up helper functions that the team will use later for validation and final submission. My work focused on making sure that once the model is trained, we will be able to evaluate it correctly and create a valid Kaggle CSV file.

The assignment requires us to predict the class labels for the unseen samples in `test_data.parquet`. Since Kaggle did not provide a `sample_submission.csv` file, I created our own temporary submission format using the unique `Sample_ID` values from the test data. This is important because each `Sample_ID` appears multiple times in the test dataset due to the time-step structure, but the final Kaggle submission should only contain one prediction per `Sample_ID`.

I loaded the following files:

```
train_labels.csv
test_data.parquet
```

I used `train_labels.csv` to check how the class labels are represented and to confirm the expected label column. I used `test_data.parquet` to extract the unique test `Sample_ID` values that must appear in the final Kaggle submission file.

I then created a temporary submission table with the following columns:

```
Sample_ID
class_label
```

The `class_label` column was temporarily filled with 1 for every sample. These are not real predictions. They are only placeholder values used to test that the submission format is correct. I also checked that the temporary submission had no missing values and no duplicated `Sample_ID`s.

I also created a train/validation split function. This function will be used later after the feature engineering step is complete. It takes the engineered feature table and the label table, merges them using `Sample_ID`, separates the input features from the target labels, and then splits the data into training and validation sets. The split uses an 80/20 ratio, meaning 80% of the data is used for training and 20% is used for validation. It also uses stratification so that the class distribution remains similar in both sets.

Mashudu Mavhila

Another important function I added is the model evaluation function, `evaluate_model`. This function takes a trained model, validation features, and validation labels. It then uses the model to make predictions on the validation set and calculates the Macro-F1 score. It also prints a classification report and confusion matrix. This is useful because the Kaggle competition evaluates performance using Macro-F1, so our local validation should use the same metric.

I also created the `create_kaggle_submission` function. This function will be used near the end of the project after Member 3 trains the final model and Member 2 provides the final test feature table. The function takes the trained model and the test features, generates predictions, creates a submission DataFrame with `Sample_ID` and `class_label`, and saves it as a CSV file called `kaggle_submission.csv` in the Submission folder. This file will be uploaded to Kaggle.

Work Completed

1. Checked the Kaggle competition page.
2. Confirmed that no `sample_submission.csv` file was provided.
3. Loaded `train_labels.csv` and `test_data.parquet`.
4. Checked the column names and previewed both files.
5. Created a temporary submission format using unique test `Sample_IDs`.
6. Checked for missing values and duplicated `Sample_IDs`.
7. Created a train/validation split function.
8. Created a Macro-F1 model evaluation function.
9. Created a final Kaggle submission export function.
10. Added explanations in the notebook so the rest of the team can follow the work.